

Ingeniería de software 2

En esta materia vamos a dar 6 temas principales, estos son:

1. **Gestión o administración de proyectos**
2. **Diseño e implementación de software**
3. **Verificación y validación**
4. **Mantenimiento de software**
5. **Gestión de configuración**
6. **Conceptos de auditoría y peritaje.**

Comunicación: Tenemos Elevator Pitch y la creación de power point.

Para crear un power point, las pantallas deben ser limpias, donde - es +, un experto en marketplace dice que hay que usar la regla 10/20/30, dónde presentación no debe tener más de 10 diapositivas, no durar más de 20 minutos y que la letra no sea menor a 30 así se ve bien por más que la persona esté lejos. Las letras sin animar, usar fondos claros y se aconseja dejar siempre un espacio para preguntas al final.

Detalles estéticos: Colores oscuros para el texto y claro para el fondo, mantener el formato del texto, usar imágenes como ayuda visual solo cuando el concepto resultaría muy difícil de entender si no estuvieran, no excederse de las animaciones y transiciones.

¿Cómo expresar los requerimientos en un documento formal?

Este documento se denomina ConOps y es un documento dirigido a los usuarios que describe las características del sistema desde el punto de vista del usuario. El estándar no especifica técnicas exactas, sino una guía de referencia que debe respetarse.

También tenemos un documento denominado SRS que define los requerimientos del software. Un documento SRS es la especificación de las funciones que debería realizar el producto de software terminado. Se aconseja que sea realizado con la intervención de la parte desarrollada y la parte del cliente.

TEORÍA 2: Gestión de la configuración del software

Cuando estamos construyendo el software suelen suceder cambios que hay que tener en cuenta sobre todo porque esos cambios pueden aumentar la confusión del equipo de trabajo y del cliente. Si no se estudian esos cambios y se analiza en dónde nos va a afectar podemos perdemos calidad del producto final porque nos generarían muchos errores si no se tratan como deberían.

Los elementos de la configuración del software:

- Especificación del sistema
- Plan del proyecto de software
- Especificación de diseño
- Listados del código fuente
- Planificación y procedimientos de prueba
- Casos de prueba y resultados registrados
- Manuales de operación e instalación
- Programas ejecutables
- Descripción de la base de datos, esquemas modelos y datos iniciales
- Manual de usuario
- Documentos de mantenimiento
- Estándares y procedimiento de ingeniería de software.

Los cambios se pueden producir en cualquier momento por lo tanto la GCS(Gestión de configuración del software) sirve para: Identificar el cambio, controlar ese cambio, garantizar que el cambio se implemente adecuadamente, también para informar el cambio a todos aquellos que puedan ser afectados.

Una línea base es un concepto que nos ayuda a controlar los cambios. Es un punto en donde tanto el cliente como los desarrolladores se ponen de acuerdo en lo que hay que hacer para el proyecto, esto sirve como base para el desarrollo y después de eso solo pueden cambiarse las cosas contenidas en la línea base a través de procedimientos formales de control de cambio.

El proceso de la GCS consta de 5 partes:

1. Identificación de los objetos/elementos en la GCS: Cada elemento se va a identificar con un nombre, que va a ser una cadena de caracteres sin ambigüedad acompañada de una descripción.
2. Control de versiones: Combina procedimientos y herramientas para gestionar los cambios realizados en un archivo (las versiones) a lo largo del tiempo, con el fin de poder recuperarlos en caso de ser necesario, pero las nuevas versiones deberían traer nuevas funcionalidades, mejores rendimientos, reparan errores anteriores del sistema, etc. De acá van saliendo ramificaciones de versiones.
3. Control de cambios: A lo largo del proyecto los cambios son inevitables y el control es vital para el desarrollo del mismo, se suelen combinar procedimientos humanos y las herramientas adecuadas para proporcionar un mecanismo para el control del cambio.
4. Auditoría de la configuración: La identificación y el control de versiones y el control de cambio ayudan al equipo de desarrollo de software a mantener un orden, pero sólo se garantiza hasta que se ha generado la orden de cambio. Para asegurarnos que se han realizado correctamente necesitamos revisiones técnicas formales y auditorías de configuración.
5. Generación de informes de estado de la configuración: responde las preguntas ¿Qué pasó? ¿Quién lo hizo? ¿Cuándo pasó? ¿Qué más se vio afectado?

Parte 2 clase 2:

Un proyecto es un esfuerzo temporal que se lleva a cabo para realizar un producto, un servicio o un resultado único. Podemos decir que es la división de un problema inicial en partes más pequeñas y sencillas, esto nos permite manejar la complejidad del software a desarrollar. Se basa en cuatro aspectos: planificación, dirección, organización y control.

Las características de un proyecto son que:

1. **Son temporales**, tienen un inicio y un fin. El fin puede llegar porque se alcanzaron los objetivos del proyecto o cuando queda claro que estos objetivos no se pueden alcanzar.
2. **Tienen un resultado** que puede ser el producto, un servicio o un resultado único.
3. **Tienen una elaboración gradual**, se desarrolla en pasos.

La administración efectiva de un proyecto de software se enfoca en las 4P (Personal, producto, proceso y proyecto) Están ordenados por orden de importancia, el recurso humano es la parte más importante ya que es la que hace el esfuerzo, por eso se debe seleccionar interesados, determinar sus habilidades y gestionarlo para lograr un proyecto exitoso. En segundo lugar está el producto que es intangible y a veces es difícil ver el progreso. En tercer lugar está el proceso, proporciona el marco de trabajo donde se establece el plan detallado para el desarrollo del software. Por último viene el proyecto, que debe ser planeado y controlado para manejar su complejidad.

Una mala gestión de proyectos puede llevar a el incumplimiento de los plazos, incremento de los costos y mala calidad en el producto final

Entonces los elementos claves para la gestión de proyectos son las métricas, las estimaciones, la planificación, análisis de riesgos y realizar un seguimiento y control. Por lo tanto la gestión del proyecto cumple todo el proceso de desarrollo, desde el inicio hasta el último mantenimiento antes de que el producto quede obsoleto y se deje de mantener.

♥ Planificación.

La planificación especifica qué debe hacerse, con qué recursos y en qué orden, lo que nos determina las tareas a hacer y cual es la secuencialidad que existe entre ellas para poder estimar el tiempo y el costo del proyecto.

Nos enfocamos en 2 tipos de planificación: •Planificación organizativa: Tiene que ver con los recursos humanos con los que contamos o que tenemos que contar para el proyecto, que es la parte más importante, por lo tanto esta planificación es un factor crítico para el éxito o fracaso de nuestro proyecto. Para ser efectivo el equipo debe organizarse de manera que se maximicen las habilidades y capacidades de cada persona.

Hay 7 factores que se deben considerar cuando se planea una estructura de equipo:

1. Dificultad del problema a resolver
2. Tamaño del programa resultante
3. Tiempo que el equipo permanecerá unido
4. Grado en el que puede dividirse en módulos el problema a resolver
5. Calidad y confiabilidad requerida por el sistema a construir
6. Rigidez de la fecha de entrega
7. Grado de sociabilidad o comunicación requerido en el proyecto

Un punto crítico a tener en cuenta es la comunicación del grupo, es algo prioritario y muy importante, ya que los programadores pueden funcionar mejor y tener una mayor productividad si cuentan con un entorno de trabajo provisto con recursos necesarios y áreas de comunicación adecuadas.

Existen 3 organigramas de equipos genéricos:

- Paradigma abierto: No tiene un jefe permanente, se nombran coordinadores de tareas a corto plazo y son sustituidos cuando cambia la tarea, las decisiones son tomadas por consenso. Se recomienda para problemas difíciles.
- Paradigma síncrono: Tiene un jefe definido que coordina tareas específicas y jefes secundarios para subtareas. La resolución de tareas es una actividad del grupo, pero la implementación de soluciones se reparte entre subgrupos, las decisiones son tomadas por consenso. Tiene mayor probabilidad de éxito en problemas complejos
- Paradigma cerrado: El jefe del equipo se encarga de la resolución de problemas a alto nivel y la coordinación interna del equipo. Se recomienda para tareas rápidas y manejo de problemas sencillos
- Paradigma aleatorio: estructura un equipo de manera holgada y depende de la iniciativa individual de los miembros del equipo. Los miembros se organizan entre ellos y no se requiere de un líder definido. Las trayectorias de comunicación pueden ser cualquiera de los otros tres paradigmas.

me voy a preparar las cosas para el fichaje bella, sigo cuando vuelvo ♥
Okis amorcito♥ Te amo y mucha suerte!

TEORÍA 3: Riesgos

Parte 1 clase 3:

¿Qué es un riesgo? Un riesgo es un evento que tiene consecuencias negativas en nuestro proyecto

Gestión de Riesgos

Los gerentes deben determinar si durante el desarrollo o mantenimiento pueden presentarse eventos no deseados, y hacer planes para evitar esos eventos, y si no se pueden evitar minimizar sus consecuencias.

Esto nos indica que lo que tenemos que hacer es anticiparnos, para que los eventos no obstaculicen nuestro proyecto o lo cancelen.

Los riesgos son: “lo que ocurrirá en un futuro” (riesgos que pueden hacer que fracase el proyecto), “cómo afectarán los cambios al desarrollo” (cambios en la tecnología, cambios de requisitos, etc) , “a la elecciones”(cantidad de gente, que herramienta, cuánta importancia hay que darle a la calidad, etc).

Estrategias para afrontar los riesgos:

Reactivas (Indiana Jones): somos conscientes que hay riesgos pero no hacemos nada para prevenirlo, sino que reaccionamos frente al problema y gestionamos las crisis

Proactivas: soy consciente del riesgo que existe y género estrategias de tratamiento para ver que puedo hacer para que no me afecte tanto

El riesgo lo definimos por su incertidumbre (porcentaje de probabilidad de que ese riesgo se presente) y su pérdida (que sucede en el proyecto si el riesgo aparece). Vamos a llamar **probabilidad** a la incertidumbre e **impacto** a la pérdida

Categorización de los riesgos:

-Proyecto: cosas relativas al desarrollo (ej: los usuarios cambian los requerimientos, necesitas más personal, etc)

-Producto: cosas que se relacionan al producto final (ej: faltan funcionalidades, cambia el lenguaje, etc)

-Negocio: cosas que se relacionan con cosas más de la empresa o algún tema legal (ej: cambio de gerente, etc)

Se pueden separar los riesgos en Genéricos y Específicos, Los genéricos son riesgos que son comunes o se pueden presentar en todos los proyectos en general. Además se pueden dividir en conocidos (riesgos conocidos por la experiencia en muchos trabajos), los predecibles (ej: cambio de requerimientos) y los impredecibles (ej: la pandemia). Los Riesgos específicos son los riesgos que son del dominio del problema, también se separan en conocidos (los que conocen el dominio, los conocen), predecibles e impredecibles.

Proceso de Gestión de Riesgos

1. Identificación de Riesgos
2. Análisis de los riesgos
3. Planeación de Riesgos
4. Supervisión de Riesgos

Este proceso es iterativo durante todo el desarrollo del proyecto.

Parte 2 clase 3:

Identificación de Riesgos:

Primero debemos seleccionar los “verdaderos riesgos”, nos puede servir la “lista de comprobación de elementos de riesgo” (tecnológicos, personales, organizacionales, herramientas, requerimientos o de estimación) o también se usa una tormenta de ideas. Hay algunas preguntas genéricas que nos ayudan a identificar los riesgos, además el grado de riesgo es proporcional a la cantidad de respuestas negativas. A cada riesgo debemos asignarle una categoría (de proyecto, de producto o de negocio) y una descripción.

Análisis de Riesgo:

De cada riesgo debemos identificar su probabilidad y su impacto. Ya con esto armamos la tabla de riesgo que debe tener:

| Nombre del Riesgo | Categoría | Probabilidad | Impacto |

Para la probabilidad se debe establecer una escala en porcentajes de bastante improbable a bastante probable, hay que recordar que ningún riesgo puede tener el 100% porque no sería un riesgo sino un problema existente

Para el impacto se debe poner: **1** (catastrófico: cancelación del proyecto), **2** (serio: retrasos, reducción de rendimiento, grandes excesos de costo), **3** (Tolerable: posible retraso, reducción mínima de rendimiento, exceso en costo), **4** (insignificante: mínima influencia en el proyecto).

Una vez puesto todo, se ordena por probabilidad e impacto y se traza una línea de corte, un número exacto de riesgos según el proyecto (Boehm recomienda 10), pero deben ser manejables. Los riesgos que queden por encima de la línea de corte son los que tendremos en cuenta, los que queden por debajo se re evalúan y van a tener prioridad de segundo orden pero no les vamos a prestar atención. Se consideran importantes los riesgos con gran impacto de probabilidad moderada a alta y los riesgos de poco impacto con gran probabilidad.

Planeación de Riesgos

Evitar el riesgo: se diseña algo para evitar que ocurra el evento

minimizar el riesgo: reducir la probabilidad de que el evento ocurra

plan de contingencia: se sabe que va a aparecer el evento, y preparar un plan para minimizar sus consecuencias

se debe tener en cuenta el costo de la aplicación de las estrategias.

Se prepara una tabla de la siguiente forma: | Riesgo | Estrategia |

Supervisión

Los riesgos se deben monitorizar en todas las etapas del proyecto.

- En cada revisión se estudia cada riesgo por separado.
- Se debe evaluar si funciona el plan que hicimos.
- También se puede cambiar la probabilidad y el impacto. y por ende cambian los riesgos que están por encima de la línea de corte
- Ver si existen nuevos riesgos

Teoría 4: Planificación temporal y diseño.

La planificación temporal es una actividad que distribuye el esfuerzo estimado a lo largo de la duración prevista del proyecto. Es prioridad lograr estimar el esfuerzo y la duración del proyecto de manera óptima y definir las tareas críticas (que son aquellas cuyo retraso genera un retraso en todo el proyecto) y los hitos (Que es algo que se espera que este hecho para algún momento en específico).

En métodos de planificación temporal vemos PERT y CPM, PERT pone en dibujos y perspectiva de tiempo, solo que en el primer caso se usa para proyectos más grandes y dibujar la red de tareas con tiempos más optimistas y pesimista, porque es un método probabilístico, en cambio CPM es un método que se usa para proyectos con menos incertidumbre porque es determinístico. Entonces se decidió tomar lo mejor de ambos métodos y definir uno nuevo que

se denomina Método del Camino Crítico que consta de 6 puntos. 1 Establecer lista de tareas 2 Fijar dependencia y duración 3 construir la red 4 Numerar los nodos 5 Calcular la fecha temprana y tardía de cada nodo 6 Calcular el camino crítico que une las tareas críticas.

Diseño del software

El diseño es una representación de lo que se va a construir. El diseño es el núcleo técnico de la ingeniería de software, ya que es cuando transformamos el problema en una solución, acá es donde fomentamos la calidad.

El diseño se divide en 4 tipos: Diseño de datos, diseño arquitectónico, diseño de interfaz y diseño a nivel componentes.

- ❖ Diseño de datos: Es donde se transforma el modelo del dominio obtenido en el análisis en estructuras de datos, objetos, relaciones, etc
- ❖ Diseño arquitectónico: Define la relación entre los elementos estructurales del soft y establece los estilos arquitectónicos a usar, los patrones de diseño, etc
- ❖ Diseño a nivel de componente: Transforma los elementos estructurales de la arquitectura de software en una descripción procedimental de los componentes de software.
- ❖ Diseño a nivel de interfaz: Describe la forma de comunicación dentro del mismo sistema, con otros sistemas y con las personas .

Para evaluar nuestro diseño tenemos que asegurarnos que se implementen todos los requisitos explícitos del modelo de análisis y ajustarse a todos los requisitos implícitos que desea el cliente, deberá ser una guía legible y comprensible para aquellos que generan código y para aquellos que dan soporte al software y deberá proporcionar una imagen completa del software.

Para construir un buen diseño tenemos 8 criterios

1. Debe tener una estructura arquitectónica que se haya creado mediante patrones de diseño reconocibles, que esté formado por componentes con buen diseño y se implemente en forma evolutiva.
2. Debe ser modular
3. Debe contener distintas representaciones
4. Debe conducir a estructuras de datos adecuadas y que procedan de patrones de datos reconocibles
5. Debe conducir a componentes que presenten características funcionales independientes.
6. Debe conducir a interfaces que reduzcan la complejidad de las conexiones entre los módulos y con el control externo
7. Debe derivarse mediante un método repetitivo y controlado por la información obtenida durante el análisis de los requisitos del software
8. Debe representarse por medio de una notación que comunique de manera eficaz su significado

Los conceptos de diseño son: Abstracción, arquitectura, patrones, modularidad, ocultamiento de información, independencia funcional, refinamiento y re-fabricación.

Teoría 5: Interfaz de Usuario.

¿Qué es el diseño de UI? diseño de computadoras, aplicaciones, sitios web, etc. Enfocado a la experiencia de usuario y a la interacción que realiza. Es una actividad multidisciplinar que involucra varias ramas del diseño y conocimiento.

El objetivo de la UI, es poder lograr la mejor interacción con el usuario, que los usuarios puedan aprender lo más rápido posible el funcionamiento del software. Las herramientas principales son la gráfica, los pictogramas, lo estético, la simbología, pero sin afectar el funcionamiento técnico eficiente!

Tenemos que saber las preferencias de las personas, que sirven para definir los conjuntos de principios de usabilidad. En la actualidad estudiamos solo la tecnología lo que lleva a que las personas se tengan que adaptar a las tecnologías, cuando debería ser al revés.

Una interfaz difícil de usar, hará que las personas se vayan de la página, además tenemos que tener en cuenta que las personas tienen diversidad con diferentes percepciones, comprensión y trabajos. Por eso la interfaz debe hacer que el usuario tenga un rápido acceso al contenido, pero sin pérdida de la comprensión mientras avanza

Hay que tener en cuenta: la variedad de dispositivos, la configuración del hardware y las tecnologías que deben adaptarse al usuario.

Hay 6 principio básicos:

1. Analizar y comprender las actividades del usuario
2. Realizar el diseño del prototipo en papel
3. Evaluar el diseño con los usuarios finales (puede volver al paso 2)
4. Realizar el diseño dinámico del prototipo
5. Evaluar el diseño con los usuarios finales (puede volver al paso 4)
6. Implementar la interfaz de usuario definitiva

es algo cíclico:

Análisis y modelado -> diseño de la interfaz -> creación de la interfaz -> validación -> análisis y modelado ->

UDX(experiencia de usuario): conjunto de métodos aplicados al diseño que busca satisfacer las necesidades del cliente y proporciona una buena experiencia

La información se toma de encuestas, información de ventas, información de mercadotecnia e información de charlas de apoyo al usuario, además se crea un perfil de usuario:franja de edad, experiencia, idioma, etc. También se identifica el contexto y ambiente de trabajo.

Aspecto de diseño de interfaz:

3 reglas doradas Dar control al usuario(que el sistema reaccione a sus necesidades) , Reducir la carga de memoria del usuario y lograr una interfaz consistente (que haya contexto y que sea siempre del mismo estilo), además sumamos factores humanos(percepción, memoria, razonamiento, capacitación, etc).

Usabilidad

No proviene de la estética, son mecanismos de interacción avanzados, la arquitectura se ajusta a las necesidades de las personas que la van a usar. es parte de lo semántico del software y

las necesidades de las personas.”es una medida de cuán bien un sistema de cómputo facilita el aprendizaje, ayuda a recordar lo aprendido, reduce la probabilidad de errores, les permite ser eficientes y los deja satisfechos con el sistema”. Hay preguntas para determinar su usabilidad. principios de usabilidad:

1. Diálogo simple y natural: forma en la que se debe llevar a cabo la interacción con el usuario
2. Lenguaje del usuario: usar el lenguaje del usuario
3. Minimizar el uso de memoria del usuario: evitar que el usuario esfuerce su memoria
4. consistencia: que no existan ambigüedades visuales ni tecnológicas
5. Feedback: Respuesta gráfica o textual en la pantalla
6. Salidas evidentes: tener una forma identificable y accesible de salida
7. Mensajes de error: información que se brinda cuando se produce un error y formas se salir de esa situación
8. Prevención de errores: evitar que el usuario llegue al error
9. Atajos: dar alternativas de manejo para que sea más cómodo y amigable
10. Ayudas: asistencia al usuario.

estilos de interfaces: de comandos, de selección de menú, gráfica de usuarios, relleno de formularios, manipulación directa, reconocimientos de voz e inteligente

Presentación de la información en la pantalla: Esto es más de estética, puede ser de forma directa con datos o de forma gráfica

colores: no más de 4 o 5 colores, usar código de colores, usar grupo de colores

soporte al usuario: Mensajes del sistema para acciones del usuario, ayudas en línea y documentación

Interfaces accesibles: deben respetar normas universales, deben poder ser accedidas por cualquier usuario independientemente de sus condiciones físicas o mentales.

Teoría 6: Diseño arquitectónico del software.

Define la relación entre los diferentes elementos estructurales para lograr los requisitos del sistema. Por eso decimos que es el que se encarga de identificar los subsistemas dentro del sistema y establecer el marco de control y comunicación entre ellos.

Para seguir hay dos cosas que tenemos que definir

Subsistema: es un sistema en sí mismo cuyo funcionamiento no depende de los servicios proporcionados por otros.

Módulo: componente de un subsistema que proporciona uno o más servicios a otros módulos y a su vez utiliza servicios de otros módulos. No se los considera un sistema independiente.

La arquitectura afecta directamente a los requerimientos no funcionales, los más críticos son el rendimiento, protección, seguridad, disponibilidad, mantenibilidad.

- Para apuntar al rendimiento se deben agrupar las operaciones críticas en un grupo reducido de subsistemas, generando componentes más grandes y de grano grueso con poca comunicación.

- Con respecto a la seguridad se puede usar una arquitectura en capas, poniendo los recursos más críticos en las capas más internas.
- Para la protección lo ideal sería que las operaciones relacionadas con la protección estén en un único subsistema (o un grupo lo más pequeño posible), ya que si hay algún problema los atacamos en un solo lugar y no disperso en todo el sistema
- Para la disponibilidad se deberá diseñar un sistema con componentes redundantes, ya que si falla un componente podemos reemplazarlo sin detener el sistema.
- Para la mantenibilidad, hay que ir en contra de otras cosas que dijimos, debe diseñarse con componentes autocontenidos de grano fino que puedan modificarse con facilidad. Hay que poner esto en la balanza con el rendimiento ya que son contrarios.

Existen 4 aspectos que tenemos que tener en cuenta con respecto al diseño arquitectónico

1. **Organización del sistema:** Representa la estrategia básica usada para estructurar el sistema, esto implica el intercambio de información de manera efectiva esto puede ser a partir de usar todos los datos compartidos almacenados en una base de datos central o que cada subsistema tenga su información y se intercambia entre subsistemas. Para esto tenemos patrones arquitectónicos, repositorios, cliente-servidor, capas o combinaciones entre ellas.
 - a. **Patrón repositorio:** En la mayoría de los sistemas que usan grandes cantidades de datos se organiza una base de datos compartida (repositorio) donde todos los datos son accedidos por todos los subsistemas del sistema. Es muy eficiente para compartir grandes cantidades de datos.
 - b. **Patrón cliente-servidor:** El sistema se organiza como un conjunto de servidores que dan servicios a unos clientes que utilizan esos servicios.
 - c. **Patrón de arquitectura en capas:** Cada capa presenta un conjunto de servicios a sus capas adyacentes. Es muy portable y resistente a cambios, además de soportar el desarrollo incremental. Una capa puede ser reemplazada siempre que mantenga la interfaz, solo que es difícil de estructurar.
2. **Descomposición modular:** Una vez organizado el sistema a los subsistemas los podemos dividir en módulos, se pueden aplicar los mismo criterios que vimos en la organización pero la descomposición modular es más pequeña y permite utilizar otros estilos alternativos. Vemos dos estrategias: Descomposición orientada a flujo de funciones y descomposición orientada a objetos.
 - a. **Descomposición orientada a flujo de funciones:** Los datos fluyen de una función a otra y se transforman a medida que pasan por las funciones hasta llegar a ser los datos de salida.
 - b. **Descomposición orientada a objetos:** Estructura al sistema en un conjunto de objetos débilmente acoplados y con interfaces bien definidas.
3. **Modelos de control:** Una vez que tenemos nuestra arquitectura a nivel de módulos hay que controlarlos que se puede dividir en 2
 - a. **Control centralizado:** Un subsistema tiene la responsabilidad de iniciar y detener a los otros subsistemas.
 - b. **Control basado en eventos:** Cada subsistema responde a eventos externos al subsistema.

4. **Arquitecturas de los sistemas distribuidos:** Nos permiten compartir recursos, tener concurrencia, escalabilidad, tolerancia a fallos, etc
 - a. **Arquitectura multiprocesador:** El sistema está formado por varios procesos que pueden o no ejecutarse en procesadores diferentes. Es común en sistemas grandes de tiempo real que recolectan información, toman decisiones y envían señales para modificar el entorno.
 - b. **Arquitectura cliente-servidor:** Los clientes y servidores son procesos diferentes que seguramente corren en distintos equipos. Un servidor puede brindar varios servicios a diferentes clientes a la vez (Si así está programado)
 - c. **Arquitectura de componentes distribuidos:** Diseña el sistema como un conjunto de componentes que brindan una interfaz de servicios que ellos suministran, y existen otros componentes que solicitan esos servicios, es como cliente servidor pero sin la distinción entre clientes y servidores
 - d. **Computación distribuida interorganizacional:** Una organización tiene varios servidores y reparte su carga computacional entre ellos

Una vez terminado el diseño se pasa a la etapa de codificación y documentación, la documentación del código es un conjunto de descripciones escritas que explican al lector qué hace el programa y cómo lo hace. La documentación se divide en dos clases: Documentación interna y externa, la documentación interna es concisa, escrita a nivel de programador, está dirigida a quienes lean el código fuente, incluye información del algoritmo, estructuras de control, etc en cambio la documentación externa se prepara para quienes tal vez nunca verán el código real.

Teoría 7: Métricas de Software.

Las métricas y las estimaciones son elementos claves de la gestión de proyectos

Métricas

Los objetivos son:

- **Entender** que ocurre en el desarrollo y mantenimiento
- **Controlar** lo que ocurre en nuestro proyecto
- **Mejorar** nuestros procesos y productos
- **Evaluar** la calidad

definiciones:

- **Medida:** indicación cuantitativa de algún atributo de un proceso o un problema
- **Medición:** el acto de medir o determinar la medida. Calcular de alguna forma la medida.
- **Métrica:** lo que utilizo para lograr una medida, aplicó una métrica para realizar la medición y lograr una medida.
- **Indicador:** define si estamos bien encaminados en el proceso.

Las métricas pueden ser utilizadas para que se tomen las mejores decisiones. Ya que sirven para medir: Procesos, Proyectos o Productos

Métrica de proyecto:

- ❖ tiempo -> ajustes en el calendario
- ❖ riesgos -> rastrear riesgos
- ❖ errores que se generan -> descubrir áreas de problemas
- ❖ personas que necesito -> ajustar flujo de trabajo o tareas/ evaluar habilidad del equipo
- ❖ etc -> valorar el estado de un proyecto en marcha

Métrica de Procesos

- ❖ Propósitos estratégicos:
 - recopilación
 - intención(indicadores para mejorar el proceso)
- ❖ Uso:
 - Incorporar sentido común
 - retroalimentación
 - no usar métricas para valorar individuos
 - establecer metas claras
 - no excluir métricas

Para definir métricas del proceso hay que definir métricas del producto

Métricas del Producto

- Dinámicas:
hechas para el programa en ejecución, relacionadas a la eficiencia y fiabilidad de un programa. ej:
- Estáticas:
hechas de representaciones del sistema. Ayudan a valorar la complejidad, comprensibilidad y mantenibilidad. ej: fan-in/fan-out, longitud del código, complejidad ciclomática, etc

(existen las métricas Orientada a Objetos)

Algunas métricas del producto:

Líneas de Código (LDC): métrica más común para el tamaño del producto, es una técnica postmortem. Con esta métricas orientadas al tamaño se puede obtener: productividad, calidad y costo.

Punto función: mide la cantidad de funcionalidad de un sistema descrito en una especificación, es decir, mediante un sistema de puntaje a ciertos puntos o preguntas se establece el estado de un sistema. También se puede obtener la productividad, calidad y costo.

GQM o OPM(objetivo, pregunta, métrica): métrica para medir cierto objetivo, tiene un nivel conceptual en el que se define un objetivo, el nivel operativo se crean preguntas para verificar que se cumpla el objetivo y el nivel Cuantitativo donde se asocian métricas

a cada pregunta para responderlas de modo cuantitativo. Es útil para decidir que medir, esta orientado a metas y es flexible

Técnicas postmortem: deja una línea base para futuras métricas, ayuda al mantenimiento conociendo la complejidad, tamaño, etc y ayuda a los procesos de reingeniería.

Estimaciones

Son técnicas que permiten dar un valor aproximado a algo que intento medir y no se puede hacer de manera exacta. Hay que tener mucho cuidado cuando utilizarlas. El resultado se obtiene de varias técnicas de estimación.

Se pueden estimar recursos, costo, tiempo. Hay que tener en cuenta que no es una ciencia exacta y se requiere de experiencia para estimar. Influyen factores como el tamaño de un producto o la estructuración de un proyecto.

Estimación de Recursos: Recursos humanos(personal: número, habilidad y ubicación), Recursos del entorno (hardware, internet, etc) y Recursos de Software(comprar componentes hechos, componentes previos, hacer nuevos componentes). Tenemos que dejar muy claro qué recursos necesitamos.

Estimación de Costos: Los 3 parámetros son: Esfuerzo(desarrolladores), Hardware/Software(mantenimiento, internet, plataforma) y Viajes. Costo no es precio, costo es lo que nos va a costar realizar el proyecto, para sacar el precio se tienen en cuenta además otras cosas.

Estimación de Tiempo: Calcular un tiempo más temprano y uno más tardío.

Técnicas de Estimación:

- Juicio Experto: consultamos a varios expertos que estiman, comparan y discuten
- Técnica Delphi: sucesivas rondas de consultas a expertos pero son anónimas, esto hace que la opinión de los distintos expertos no se vea influenciada, después nosotros hacemos un consenso.
- división de trabajo: preguntamos a la gente que está más abajo en la jerarquía y se va subiendo a medida que se va consensuando

Modelo Empírico de estimación:

COCOMO II: se creó por recopilar datos de un gran número de proyectos, tiene bases en el COCOMO I o 81, está compuesto por 4 modelos:

- modelo de composición de aplicación: modela el esfuerzo requerido y se basa en una estimación de puntos de aplicación (estimación del número de pantallas, informes, módulos, líneas de código)
- modelo de diseño temprano: se usa para las primeras etapas del proyecto, supone que se acordaron requerimientos del usuario y que ya está empezando. La meta es tener una estimación rápida y aproximada de los costos
- modelo de reutilización: se emplea para estimar el esfuerzo al integrar código reutilizado o generado, el modelo estima el número de personas/mes necesarias para integrar el código
- modelo de post arquitectura: se usa cuando esta disponible el modelo arquitectónico y se hacen estimaciones de cada parte del sistema

Teoría 8: Tipos de pruebas de software

El software falla cuando no hace lo que especifican los requerimientos.

Una de las causas del fallo del software es el error de código que se puede clasificar en algorítmicos (Ej no inicializar variables), de sintaxis (Ej confundir un 0 por una o), de precisión (fórmulas no implementadas correctamente), de documentación (Ej la documentación no está acorde con lo que hace el software), de sobrecarga (Ej el sistema funciona bien con 100 usuarios pero no con 1.000), de coordinación o sincronización (Ej la comunicación entre procesos falla), de rendimiento (El tiempo de respuesta no es adecuado), de recuperación (No volver a un estado normal luego de una falla), etc

Existe la clasificación octagonal de defectos

TIPO DE DEFECTO	SIGNIFICADO
Función	Afecta la capacidad, interfaces
Interfaz	Afecta a la integración con otros componentes
Comprobación	Afecta la lógica del programa
Asignación	Afecta la estructura de los datos
Sincronización	Involucra sincronización de recursos compartidos y de tiempo real
Construcción	Ocurre debido a problemas en repositorios, gestión de cambios o control de versiones
Documentación	Afecta a publicaciones
Algoritmo	Involucra la eficiencia o exactitud de un algoritmo

Las pruebas son exitosas cuando se descubren errores, ya que es casi imposible que el software funcione perfectamente durante el desarrollo, por lo tanto hay que descubrir los errores antes de que el software salga del ambiente de desarrollo.

Tenemos dos tipos de prueba

- ★ Pruebas de caja blanca/abierta/cristal: Mira dentro del código para encontrar errores.
- ★ Pruebas de caja negra/cerrada: Ingresa datos y mira que es lo que me devuelve el software, sin saber como es el código por el que pasaron. En caja negra existe la prueba de partición equivalente (Como la de Obj 1), una vez que tenemos todas las particiones hechas hacemos el Análisis de valores límite ALV (También igual que en Obj1) que quiere decir que si tengo un rango que va desde A a B tengo que probar a, b, < a y > b.

Teoría 9: Estrategias de pruebas de software

Enfoque estratégico de pruebas, proporciona una guía que describe los pasos a seguir, cuando se planean y llevan a cabo, cuánto esfuerzo, tiempo y recurso se requerirán. Nos da como resultado una planificación de las pruebas, un diseño de los casos de prueba, ejecución de las pruebas y recolección y evaluación de datos resultantes.

Es un conjunto de actividades que se planean con anticipación y se realiza de manera sistemática, incluyen tácticas y métodos específicos de diseños de pruebas. Debe incluir pruebas de bajo y alto nivel. Esto es parte de la **Verificación y Validación** de un software
verificación: que el software implementa correctamente una función específica. Se debe corroborar que el software satisface los requerimientos funcionales y no funcionales
Validación: que el software corresponde con los requisitos del cliente, es decir, satisface las expectativas del cliente.

Hay una espiral donde están:

1. prueba de unidad
2. Pruebas de Integración
3. pruebas de validación
4. pruebas de sistema

Pruebas de Unidad

Lo que hacemos es probar cada unidad o módulo por separado, ingresando datos y viendo si la respuesta que da es la esperada.

Se puede probar la interfaz del módulo, las estructuras de datos locales, las condiciones límite y ejercitar todos los caminos independientes.

Errores más comunes detectados: cálculos incorrectos, comparaciones erróneas o flujos de control inapropiados

Procedimiento: se debe desarrollar para cada prueba un software que controle y resguarde.

Un **controlador** es un “programa principal” que envía los datos a la unidad probada y recibe los resultados. Los **resguardos** reemplazan los módulos subordinados de la unidad a probar.

Estos dos dan una sobrecarga de trabajo si no hay un alto grado de cohesión.

Pruebas de Integración

Se toman los componentes ya probados y se los combina según el diseño, en esta combinación puede pasar que los datos se pierdan en una interfaz, uno de los módulos puede tener un efecto adverso sobre otro, la combinación de subfunciones no da el resultado esperado, etc.

El programa se construye y se prueba en pequeños segmentos para que los errores sean más fáciles de aislar y corregir. la integración puede ser:

- Descendente: se integran descendiendo por la jerarquía de control, se van probando con resguardos de cada subnivel en cada paso y luego se reemplazan por el verdadero 1 a 1, cada vez que reemplazó el resguardo por el real, debe ser probado devuelta porque se pueden generar errores, puede ser:

- en Profundidad: integrar todos los módulos de un camino de control principal (DFS)
- en Anchura: incorpora todos los módulos subordinados de cada nivel, por niveles (BFS)
- Ascendente: se empieza probando los modelos atómicos, y la integración se hace de abajo hacia arriba en la jerarquía. no se necesitan resguardos, pero si conductores. Lo que se hace es combinar los módulos, hacer el conductor, probar el grupo, borrar conductor.

Ventajas y Desventajas

Desventajas:

descendentes: la necesidad de Resguardos

ascendentes: el programa no existe hasta que agregue el último módulo.

Un enfoque sandwich o combinado suele ser el mejor

Pruebas de Regresión: cada vez que se añade un nuevo módulo el software cambia, estos cambios pueden traer problemas, por lo que deben volver a ejecutar un subconjunto de pruebas que se llevaron a cabo antes, para asegurarse que todo sigue funcionando correctamente y que no haya errores al integrar los módulos. Se puede hacer manuales o utilizando herramientas automáticas, hay 3 clases:

- ejercitar todas las funciones del sistema
- se centran en las funciones que puedan ser afectadas
- se centran en los componentes que fueron cambiados

Criticidad: se deben identificar los módulos críticos, los que:

- abordan muchos requerimientos
- tienen alto nivel de control
- es complejo o proclive a error
- tienen requerimientos de rendimientos definidos

deben probarse lo antes posible y las pruebas de regresión deben hacer foco en ellos.

Para OO:

pruebas de unidad: una clase encapsulada, los métodos son las unidades más pequeños a ser probados, se llama Prueba de Clase

pruebas de Integración: no hay una jerarquía de control, se usa la prueba basada en hebra(hilos) que integra un conjunto de clases para responder un evento, otra es la basada en uso que empieza con las clases independientes y luego las dependientes

Pruebas del sistema

Para verificar que todo se integró correctamente y que funciona bien existen una serie de pruebas:

- pruebas de Recuperación: a fallas y modo de reanudación
- Pruebas de Seguridad: mecanismos de protección
- Pruebas de Resistencia: situaciones anormales
- Pruebas de rendimiento: en tiempo de ejecución

Pruebas de Validación

Mediante una serie de pruebas se demuestra la conformidad con los requisitos. Una vez que se procede con cada caso puede darse: “están de acuerdo con las especificaciones y son aceptables” o “se descubre una desviación y se crea una lista de deficiencias”

Se incluye una revisión de la configuración.

Pruebas de Aceptación (alfa y beta): las realiza el usuario final, puede ser algo informal o una ejecución sistemática de pruebas. Beta: “clientes seleccionados las hacen sin presencia del desarrollador”

Alfa “desarrolladores con clientes antes de lanzar el producto”

se hacen en un lugar de desarrollo, en un entorno controlado, con el desarrollador observando y anotando errores, no es la versión final aún se pueden añadir funcionalidades pero se hace después de las pruebas de unidad e integración, es un sistema ya funcionando.

Beta “clientes seleccionados las hacen sin presencia del desarrollador”

Se hacen en el lugar de trabajo del cliente, no está presente el desarrollador, es una prueba en vivo en un entorno no controlado por el desarrollador. El cliente registra todos los problemas, estas pruebas son la última fase y se hacen con técnicas de caja negra, esta versión es liberada al mercado y se hacen modificaciones en base a los comentarios de los usuarios

Depuración

proceso de identificar y corregir errores, no es una prueba pero ocurre por una prueba efectiva que detectó el error o también que haya surgido un error una vez lanzado el producto.

Puede pasar que: “encontremos la causa, la corregimos y eliminamos” o “que no encontremos la causa, se debe sospechar la causa, diseñar casos de prueba(puede ser un programa adicional) para confirmar la sospecha o rastrear manualmente el error y se vuelve a la corrección de error” hay características que complican la depuración ej: es lejano en el código, síntoma desaparece temporalmente, es por error humano, etc

Pruebas en entornos especializados

a medida que se hace más complejo, se necesitan enfoques de pruebas especializados:

- pruebas de interfaces gráficas
- prueba de arquitectura cliente-servidor: de servidor, aplicación, BD, transacciones, comunicación de red
- pruebas de documentación y ayuda: revisar guía de usuario o ayuda en línea, pruebas de documentación (revisar e inspeccionar y pruebas en vivo).
- Pruebas de sistema en tiempo real: manejo de interrupciones, paralelismo, de tareas, comportamiento, inter-tareas y sistemas.

Teoría 10: Mantenimiento del software

El mantenimiento es la atención al sistema después de que se ha entregado. A esta etapa también se la conoce como “Evolución del sistema”

Esto también nos lleva al punto de evaluar cuándo es conveniente cerrar el ciclo de vida de ese sistema y reemplazarlo por otro, la decisión muchas veces tiene que ver con la complejidad de entender la documentación o un código difícil de entender, etc pero además porque esto involucra solucionar errores, añadir mejoras y optimizar lo que ya está hecho, lo que provoca costos adicionales. Se alcanza la “Barrera de mantenimiento” cuando se decide ya no continuar con el mantenimiento de un software.

Características del mantenimiento: Disminuye otros desarrollos al tener que dedicar programadores a mantener un software viejo en vez de poder crear software nuevo. Pueden existir efectos secundarios sobre código, datos, documentación. Las modificaciones pueden provocar la disminución de la calidad total del producto. Las tareas de mantenimiento generalmente provocan reiniciar las fases de análisis, diseño e implementación. Pueden involucrar entre un 40 y 70% del costo total de desarrollo.

El trabajo de mantenimiento no suele ser un trabajo atractivo, no es innovador, porque en la solución original tal vez no se veían cambios y ahora son difíciles de implementar, porque suele ser difícil comprender código ajeno sobre todo sin la documentación adecuada. Además implica volver a hacer todos los pasos que seguimos al iniciar un proyecto tales como el análisis, diseño, implementación, prueba, actualizar la documentación, distribuir e instalar la nueva versión.

Existen 4 tipos de mantenimientos:

1. **Mantenimiento correctivo:** Se diagnostican y corrigen errores.
2. **Mantenimiento adaptativo:** Modificación del software debido a cambios en el entorno, por ejemplo cambios en el hardware.
3. **Mantenimiento perfectivo:** El enfoque es mejorar el sistema y agregar nuevas funcionalidades
4. **Mantenimiento preventivo:** Se realiza antes de que se pida un mantenimiento, se utiliza para facilitar el futuro mantenimiento previniendo errores.

El más usado es el perfectivo, le sigue el adaptativo, después el correctivo y por último el preventivo.

Rejuvenecimiento del software: es un desafío de mantenimiento, intenta aumentar la calidad global de un sistema existente ayudando a prevenir la degradación de ese sistema. Nosotros vemos 4 tipos:

- ❖ **re- documentación:** Representa un análisis del código para producir la documentación del sistema, ya que puede pasar que no se entienda, que esté obsoleta o demás.
- ❖ **la reestructuración:** Consta en modificar el código para hacerlo más fácil de entender, para ordenarlo, para estructurarlo, etc
- ❖ **la ingeniería inversa:** Se usa cuando solo tenemos el código fuente, sin ningún tipo de documentación de ninguna de las etapas y se trata de generar todos los documentos necesarios a partir de ese código fuente.
- ❖ **reingeniería:** Es una extensión de la ingeniería inversa, ya que solo se nos proporciona un código, ahí se le aplica ingeniería inversa para generar documentos y

una vez que tengo todos esos documentos, se mejora ese código o se puede producir uno nuevo correctamente estructurado mejorando la calidad pero sin cambiar la funcionalidad del sistema.

Teoría 11: Auditoria Informatica

Una auditoría informática permite definir estrategias para prevenir delitos, problemas legales, verificar la seguridad en los accesos, verificar si se aplican normas de calidad, etc.

La auditoría es una actividad preventiva: el auditor SUGIERE, su trabajo es evaluar, verificar, recomendar y sugerir, pero nunca imponer

Los procedimientos varían según la filosofía y técnica de cada organización

La auditoría es un examen crítico que se realiza para evaluar la eficiencia y eficacia de una sección, organismo, software, servicio o sistema y determinar cursos alternativos de acción para mejorar lo que estamos evaluando y lograr los objetivos.

Los objetivos de una auditoría informática son: Salvaguardar los activos (Proteger Hw, Sw y recurso humano), integridad de datos (Los datos deben ser consistentes), Efectividad del sistema (El sistema debe cumplir con los objetivos de la organización), eficiencia de los sistemas (Es cumplir lo objetivos con la menor cantidad de recursos), seguridad y confidencialidad.

NO es una actividad meramente mecánica.

La auditoría puede influir en factores como el control del uso de la computadora, ver si se tiene una pérdida de capacidades de procesamiento de datos, ver si se está logrando mantener la privacidad tanto individual como de la organización, evaluar si hay pérdida de información o mal uso de la misma, toma de decisiones incorrectas, etc.

Los campos de acción son:

- Evaluación administrativa de informatica
- Evaluación de los sistemas, procedimientos y eficiencia en el uso de la información
- Evaluación de proceso de datos, de sistemas y equipos de computo
- Seguridad y confidencialidad
- Aspectos legales de los sistemas y la información.

Los auditores deben evaluar los sistemas informáticos y procesos relacionados con la tecnología(también infraestructura), para ver si es lo que la empresa necesita y ofrecer soluciones viables a cualquier problema que se haya detectado. Además, deben identificar el potencial de optimización de procesos informáticos a partir de debilidades y errores. Tienen que seguir las normas y estándares de cada empresa. El auditor tiene que controlar el funcionamiento de sistemas informáticos internos y de cualquiera que dependa de la empresa, detectar riesgos para prevenir ciberataques y desvío de datos, finalmente escribir informes detallados que incluyan los resultados de la auditora, diseñar soluciones a los problemas detectados y proponer estrategias para mejorar los sistemas. Un auditor puede trabajar para una empresa en particular o de manera independiente y ser contratado por estas.

etapas genéricas: 1_ planeación. 2 ejecución. 3 análisis de los datos recabados y condiciones observadas. 4_ Elaboración de un informe escrito y emisión de opinión.

Finalizar la auditoría: cuando se termina la auditoría se deben implementar las mejoras recomendadas y es esencial realizar un seguimiento para ver si realmente son efectivas. Además, aunque depende de la empresa, se recomienda hacer una auditoría una vez por año.

Tipos de Auditoría:

- Auditoría Informática
- Auditoría Interna: Es la realizada con recursos materiales y personas que pertenecen a la propia empresa. Es por decisión de la empresa así que puede ser disuelta cuando ellos quieran
- Auditoría Externa: Es realizada por personas ajenas a la empresa auditada. Se presupone mayor objetividad debido al distanciamiento entre auditores y auditados.
- Combinación de Interna o externa

Principios aplicados al Auditor informático

- **Principio de beneficio del Auditado:** El auditor no debe obtener beneficio propio de la empresa auditada.
- **Principio de Calidad:** el auditor debe prestar sus servicios con las posibilidades de la ciencia y medios a su alcance con total libertad respecto al uso de esos medios para el cumplimiento de su labor.
- **Principio de Confianza:** el auditor debe facilitar e incrementar su confianza mediante una actuación transparente, profesionalmente y sin alardes científicos-técnicos
- **Principio de Capacidad:** el auditor debe estar plenamente capacitado para hacer su labor, teniendo en cuenta que a los auditados les puede ser difícil ver sus recomendaciones y evaluar la precisión de las mismas
- **Principio de Comportamiento Profesional:** en las relaciones con el auditado o con terceras personas deberá actuar conforme a las normas implícitas y explícitas de dicha profesión
- **Principio de Concentración en el trabajo:** deberá evitar que un exceso de trabajo supere sus posibilidades de concentración y precisión en cada una de las tareas a realizar, ya que puede afectar a las garantías de seguridad que debe dar
- **Principio de Discreción:** deberá mantener cierta discreción en la divulgación de datos, aparentemente inocuos, que obtiene durante la auditoría
- **Principio de Economía:** deberá proteger (dentro de sus conocimientos) los derechos económicos del auditado, evitando gastos innecesarios en su actividad.

- **Principio de Formación Continua:** se impone el deber y responsabilidad de mantener actualizados constantemente sus conocimientos para las necesidades de los auditados.
- **Principio de Fortalecimiento y Respeto de la Profesión:** la defensa pasa por el fortalecimiento de la profesión, se exige un respeto por el ejercicio de la actividad y un comportamiento acorde con los requisitos exigibles para el cumplimiento de las auditorías
- **Principio de Independencia:** tener criterio propio, tanto si trabaja externamente o no a la empresa, debe tener total autonomía e independencia en su trabajo
- **Principio de Información Suficiente:** obliga al auditor a aportar de forma pormenorizada, clara, precisa e inteligible (para el auditado) los puntos y conclusiones de la auditoría.
- **Principio de Legalidad:** exige al auditor un comportamiento activo en contra de todo intento de infringir cualquier derecho positivo
- **Principio de Integridad Moral:** está ligado a la dignidad de la persona, el auditor debe ser honesto, leal y diligente en su misión, ajustarse a las normas morales de justicia y prioridad
- **Principio de Libre competencia:** exige el ejercicio en el marco de la libre competencia, es decir, se rechazan prácticas que impidan o limiten la competencia de otros profesionales
- **Principio de No Discriminación:** en su actuación previa, durante o posterior, se deberá evitar cualquier tipo de condicionante personalizado y actuar en todos los casos de la misma manera
- **Principio de no injerencia:** se deberá evitar injerencias, respetar la labor y eludir hacer comentarios sobre el trabajo de otros profesionales.
- **Principio de Precisión:** no se debe concluir el trabajo hasta estar seguro de la viabilidad de sus propuestas
- **Principio de la Publicidad adecuada:** la oferta y promoción de los servicios deben ajustarse a las características, condiciones y finalidad perseguida
- **Principio de Responsabilidad:** deberá responsabilizarse de todo lo que haga, diga o aconseje
- **Principio de Secreto Profesional:** confidencialidad y confianza, el auditor debe guardar los secretos de los hechos e informaciones que conozca durante la auditoría
- **Principio de Servicio Público:** debe incitar al auditor a hacer lo que este en su mano y sin perjudicar a su cliente, para evitar daños colosales
- **principio de veracidad:** debe tener siempre presente la obligación de asegurar la veracidad de su trabajo con respeto, corrección y secreto profesional.

Metodologías

Octave, es una especie de escalera, es una serie de pasos estructurados que identifican los activos críticos, analizan amenazas y evalúan vulnerabilidades.

el primer paso es identificar activos, el segundo analizar amenazas y el tercero determinar vulnerabilidades

Margerit, es una pirámide, se realiza de manera sistemática y exhaustiva. El primer paso es la planificación, el segundo el análisis, el tercero la evaluación y el cuarto la recomendación.

El auditor debe hacer una revisión de los controles técnicos(firewalls, configuración de servidores,etc) , de controles físicos (en el lugar, cámaras, resguardo de áreas, etc) y controles administrativos(políticas de seguridad, procedimientos, etc).

Reporte de hallazgos: se documentan de manera clara y concisa los problemas, debilidades o riesgos identificados. El informe debe proporcionar una visión general de los hallazgos, clasificados por criticidad y acompañado de recomendaciones.

Recomendaciones y Plan de acción: se brindan recomendaciones específicas y detalladas, se establece un plan de acción claro y estructurado con plazos y recursos, y luego un seguimiento y monitoreo para evaluar su progreso

Consultor informático: asesora a las organizaciones en cuestiones relacionadas con las tecnología y los sistemas de la información (orientado a la eficiencia para lograr los objetivos)

Auditor Informático: evalúa y verifica los sistemas de información y los controles internos (más orientado a la seguridad)

Perito Informático: actúa como experto en casos legales o judiciales relacionados con la informática